Plan: Validación con clasificador alternativo, diversidad genética y escalabilidad

TFM: Selección Automática de Características en Algoritmos de Clasificación Mediante Algoritmos Genéticos y Métodos de Envoltorio con Apache Spark RDD

Este documento planifica dos tareas pendientes de la fase de experimentación y deja constancia razonada de los conceptos implicados y de las decisiones tomadas:

1. **Validación del subconjunto final S^* con un clasificador distinto de GNB** (LinearSVC).
2. **Análisis adicional de diversidad genética y escalabilidad**, ampliado a estabilidad de la selección.

Ambas tareas se aplican a los tres datasets del estudio: **WDBC** ($D = 30$), **Arrhythmia** ($D = 279$) y **Gisette** ($D = 5000$).

Parte I — Marco conceptual

1. Diversidad genética

1.1 Qué es y por qué importa

La **diversidad genética** mide cuánto se diferencian entre sí los individuos de la población a lo largo de las generaciones. Es el motor de la exploración en un algoritmo genético: sin diversidad no hay material sobre el que operen el cruce y la mutación, y la búsqueda degenera en una explotación local.

Su valor para la memoria es doble:

- **Diagnóstico de convergencia prematura.** Si la diversidad colapsa demasiado pronto, el GA se atasca en un óptimo local; la población se vuelve homogénea y deja de explorar el espacio 2^D de subconjuntos. La curva de diversidad permite afirmar, con evidencia, que el GA *realmente busca* y no solo deriva.
- **Justificación de los parámetros.** El equilibrio exploración → explotación depende de la tasa de mutación $p_{\text{mut_bit}}$, el tamaño de torneo k, el elitismo E y la inicialización p_{init}. La evolución de la diversidad respalda empíricamente esas elecciones de diseño.

La diversidad se mide a dos niveles complementarios:

- **Genotipo:** diferencias en los propios vectores binarios (qué características activa cada individuo).
- **Fenotipo:** diferencias en el valor de fitness (la "calidad" observable de los individuos).

1.2 Métricas y sus relaciones

Sea p_j la fracción de la población cuyo gen j vale 1 (frecuencia alélica del locus j). Sobre la matriz booleana población × características ($P \times D$):

Diversidad por locus (Gini), normalizada a $[0, 1]$:

$$\bar{H}_{\text{Gini}} = \frac{1}{D} \sum_{j=1}^{D} 4\, p_j\, (1 - p_j)$$

Vale 1 cuando todos los loci están equilibrados ($p_j = 0.5$) y 0 cuando la población es idéntica en todos los genes.

Entropía de Shannon media por locus (en bits):

$$\bar{H}_{\text{Shannon}} = \frac{1}{D} \sum_{j=1}^{D} \left[-\, p_j \log_2 p_j - (1 - p_j) \log_2 (1 - p_j) \right]$$

Distancia de Hamming media normalizada entre pares. La distancia esperada entre dos individuos tomados al azar es:

$$\bar{d}_{\text{Hamming}} = \frac{1}{D} \sum_{j=1}^{D} 2\, p_j\, (1 - p_j)$$

Su gran ventaja es que se calcula en $\mathcal{O}(P \cdot D)$ a partir de las frecuencias p_j, **evitando el coste** $\mathcal{O}(P^2 \cdot D)$ de comparar todos los pares explícitamente — relevante para Gisette ($D = 5000$).

> **Observación clave (evita trabajo redundante):** las tres métricas anteriores son proporcionales entre sí, pues todas dependen de $\sum_j p_j(1 - p_j)$. Basta calcular **una** métrica genotípica. Se elige la **Hamming media normalizada** por su interpretación directa como distancia y por su bajo coste.

A esta métrica genotípica se añaden dos indicadores baratos y muy interpretables:

- **Número de individuos únicos:** $n_{\text{únicos}}/P$ = genotipos distintos sobre el total. Su caída señala la aparición de duplicados, síntoma directo de convergencia.
- **Dispersión de fitness (fenotípica):** std(fitness) de la población. Complementa los valores `mejor`/`medio`/`peor` que ya se registran.

1.3 Por qué se debe medir *dentro* de la ejecución del GA

El fichero de series actual (`*_series.csv`) guarda, por generación, únicamente agregados de fitness (`mejor`, `medio`, `peor`) y el número medio de genes activos (`genes_medio`). **No persiste la población completa.** En consecuencia:

- La dispersión **fenotípica** sí es reconstruible a posteriori (de hecho, `peor - mejor` ya es un proxy).
- La diversidad **genotípica** (Hamming, entropía, únicos) **no** puede reconstruirse después: requiere acceso a los vectores binarios de cada generación, que solo existen en memoria durante la ejecución.

Volcar las poblaciones completas a disco para procesarlas luego sería absurdo (para Gisette, 100 generaciones × 100 individuos × 5000 bits). La solución correcta es **instrumentar el bucle generacional** con un *hook* que calcule las métricas de diversidad sobre la población viva, en cada generación, y las anexe como columnas nuevas al CSV. El cálculo es una operación vectorizada de NumPy sobre la matriz $P \times D$ ya residente en RAM: **no añade ninguna evaluación de fitness** (lo único costoso del algoritmo) y su sobrecoste es despreciable incluso para Gisette.

Esto responde a la pregunta de planificación: **sí, tiene pleno sentido —y es prácticamente obligatorio— hacerlo dentro de la ejecución del GA**, y se aplica por igual a los tres datasets. Arrhythmia es, además, el caso "intermedio" ($D = 279$) que mejor ilustra el régimen con margen real de exploración, entre el caso trivial (WDBC) y el extremo (Gisette).

> **Caveat metodológico:** Gisette se inicializa con $p_{\text{init}} = 0.1$ (dispersa) frente a 0.5 en WDBC y Arrhythmia. Su diversidad de partida ya es, por diseño, más baja. Las curvas de diversidad **no son directamente comparables** sin normalizar respecto a la generación 0, o sin advertir explícitamente de esta diferencia de inicialización en la memoria.

2. Escalabilidad

El término engloba **dos ejes distintos** que conviene nombrar por separado para no confundirlos en la redacción.

2.1 Escalabilidad algorítmica (respecto a la dimensionalidad D)

Mide cómo evolucionan el coste y el comportamiento del GA al crecer la dimensionalidad del problema: WDBC ($D = 30$) → Arrhythmia ($D = 279$) → Gisette ($D = 5000$). Es el eje que aporta mayor valor a la tesis, porque demuestra que el método propuesto **escala de 30 a 5000 características** manteniendo resultados útiles.

Variables a comparar entre datasets: tiempo total, tiempo por generación, tiempo por evaluación, generaciones hasta converger, cardinalidad del subconjunto óptimo $|S^*|$, ratio de reducción $|S^*|/D$ y exactitud retenida.

2.2 Escalabilidad paralela (respecto a los recursos Spark)

Mide la ganancia obtenida al distribuir la evaluación de fitness con Apache Spark RDD. Métricas clásicas:

$$S(n) = \frac{T_1}{T_n}, \qquad E(n) = \frac{S(n)}{n}, \qquad S(n) \leq \frac{1}{s + \dfrac{1-s}{n}} \text{ (ley de Amdahl)}$$

donde T_n es el tiempo con n núcleos, $S(n)$ el *speed-up*, $E(n)$ la eficiencia y s la fracción serie del trabajo. Este eje **ya está parcialmente resuelto** en el proyecto mediante los experimentos `exp1_speedup_poblacion`,

`exp2_speedup_particiones`, `exp3_heatmap_speedup` y `escalabilidad_paralela.png`. El trabajo adicional consiste en **extenderlo a cada dataset**, no en empezar de cero.

3. Estabilidad de la selección

3.1 Qué es y por qué importa

Un GA es un algoritmo estocástico: dos ejecuciones con semillas distintas pueden devolver subconjuntos S^* diferentes. La **estabilidad de selección** mide cuánto se parecen entre sí los subconjuntos obtenidos en ejecuciones independientes. Una selección inestable —que cambia radicalmente con la semilla— es difícil de defender como "las características importantes del problema". La estabilidad aporta el argumento de **robustez**: existe un núcleo de características que el método identifica de forma consistente.

Se ejecuta el GA R veces con R semillas distintas **manteniendo idéntica la configuración** (mismo P, G, CV…); solo cambia la semilla. Se obtienen R subconjuntos $S_1^*, \ldots, S_R^*$ y se cuantifica su similitud.

3.2 Métricas

Índice de Jaccard medio entre pares (métrica principal; tolera subconjuntos de distinto tamaño):

$$J(S_i, S_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}, \qquad \bar{J} = \frac{2}{R(R-1)} \sum_{i<j} J(S_i, S_j)$$

Índice de consistencia de Kuncheva (corrige por el solapamiento esperado por azar). Para dos subconjuntos de tamaño k extraídos de D características y con r elementos comunes:

$$I_C = \frac{rD - k^2}{k(D-k)}$$

Su rango es aproximadamente $[-1, 1]$, con 0 = solapamiento atribuible al azar. Es **imprescindible en Gisette**, donde $k \approx 486$ sobre $D = 5000$: un Jaccard a secas sería engañoso porque ignora cuántos solapamientos cabría esperar por puro azar. (La fórmula clásica supone cardinalidad constante; con tamaños variables se promedia por pares o se complementa con Jaccard.)

Frecuencia de selección por característica:

$$f_j = \frac{1}{R} \sum_{\text{run}} c_j^{(\text{run})}$$

Permite definir un **núcleo robusto** de características seleccionadas en la mayoría de ejecuciones (p. ej. $\geq 8/10$). Es el resultado más interpretable: "estas características se seleccionan de forma consistente".

4. Validación con un clasificador distinto (LinearSVC) y el sesgo de selección

4.1 Por qué un clasificador distinto

Durante la búsqueda, el fitness usa **GNB** por su velocidad ($\mathcal{O}(N \cdot D)$), única opción viable ante las decenas de miles de entrenamientos que genera el GA. Pero un subconjunto óptimo para GNB podría estar sobreajustado a las particularidades de ese clasificador. Revalidar S^* con un clasificador de naturaleza distinta —**LinearSVC**, de frontera lineal de margen máximo— confirma que la selección **generaliza** y no es un artefacto del clasificador base. Adicionalmente, al ser LinearSVC más costoso por evaluación, sirve como segundo punto de medida de la aceleración de Spark (véase `diseño_GA.md`, §5.2).

4.2 El problema del sesgo de selección

El esquema actual calcula la exactitud mediante validación cruzada sobre **todas** las N instancias:

$$\text{Accuracy}(\mathbf{c}) = \overline{\text{CV}}(\text{GNB},\ X[:, S],\ y)$$

Si la exactitud final de S^* se reporta sobre esas mismas instancias, el valor está **optimistamente sesgado**: el GA ya las "ha visto" y ha elegido características que funcionan bien precisamente en ellas. Es el sobreajuste por selección de características, documentado por Ambroise & McLachlan (2002), y una crítica habitual en defensa.

4.3 Esquema elegido: test *held-out*

Se adopta una partición *held-out* independiente. Solo cambian **tres** cosas respecto al pipeline actual:

1. **Un split estratificado, una vez, antes del GA** (por dataset): `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)`
2. **El fitness usa solo el train** (la CV se ejecuta sobre `X_train`/`y_train`; el GA nunca ve `X_test`).
3. **Al converger, se valida S^* sobre el test no visto**: se entrenan GNB y LinearSVC en `X_train[:,S*]` y se miden en `X_test[:,S*]`.

El resto del algoritmo (operadores, diversidad, speed-up) no cambia. El coste es nulo (un split + dos `fit` finales) y no multiplica ejecuciones.

Decisiones descartadas y por qué:

- *CV sobre todas las instancias* (mantener el esquema actual): más simple, pero la validación no es independiente de la búsqueda → sesgo optimista.
- *Nested CV* (CV externa para validar + interna para buscar): máximo rigor, pero **multiplica todo el GA por el número de folds externos**; con Gisette × $R = 10$ × 5 folds se va a horas. Desproporcionado para esta escala.

  Caveat para Arrhythmia: 452 muestras y 13 clases muy desbalanceadas (algunas con 2-4 ejemplos en total). Un test del 30 % estratificado puede dejar clases minoritarias con 0-1 muestras en test. Debe vigilarse (ajustar la proporción de train o reportar honestamente que ciertas clases raras no son evaluables).

Parte II — Decisiones tomadas

Decisión	Elección	Justificación
Alcance de la escalabilidad	**Completo**: algorítmica (D) + speed-up por dataset + estabilidad de selección	Es el conjunto de evidencias más sólido para la memoria; el coste adicional es asumible ($<$ 1 h dominado por Gisette).
Nº de semillas (estabilidad)	$R = 10$ por dataset	Equilibrio estándar en la literatura de estabilidad de *feature selection*. Gisette $ $22 min; WDBC y Arrhythmia, segundos.
Organización del código	**Refactor a `src/ga_core.py`**	Hoy el GA está triplicado en los notebooks 04/05/06. Un GA único parametrizable por semilla evita duplicar el *hook* de diversidad y hace trivial el bucle de estabilidad.
Validación de S^*	**Test *held-out***	Riguroso, coste nulo, evita el sesgo de selección. Nested CV se descarta por coste; CV-sobre-todo, por falta de independencia.
Métrica genotípica de diversidad	**Hamming media normalizada** (+ `n_unicos`, `std_fitness`)	Las métricas genotípicas son proporcionales; se elige la de coste $\mathcal{O}(P \cdot D)$ e interpretación directa.
Métricas de estabilidad	**Jaccard + Kuncheva + frecuencia**	Jaccard tolera tamaños distintos; Kuncheva corrige por azar (clave en Gisette); la frecuencia define el núcleo robusto.

Parte III — Plan de implementación

Nota: esta es la planificación. La ejecución se realizará en otra sesión.

Componentes

A. `src/ga_core.py` — Extraer el bucle evolutivo a una función `run_ga(X_train, y_train, params, seed, diversity_hook=True) → {S*, series, tiempos}`. Es la base que abarata B, C y E. Los notebooks pasan a llamarla en lugar de redefinir el GA.

B. Diversidad genética (los 3 datasets, dentro del GA) — *Hook* por generación sobre la población $P \times D$; columnas nuevas en `*_series.csv`: `hamming_media_norm`, `n_unicos`, `std_fitness`. Artefactos: `results/ga_full_<dataset>_diversidad.png` por dataset + panel comparativo. Coste: 0 evaluaciones extra.

C. Estabilidad de selección ($R = 10$/dataset) — `run_ga` con 10 semillas → 10 subconjuntos S^*. Métricas: Jaccard medio, índice de Kuncheva, frecuencia por característica (núcleo $\geq 8/10$). Artefactos: `*_estabilidad.csv`, `*_estabilidad.png`.

D. Escalabilidad algorítmica — Tabla cross-dataset consolidada:

Dataset	D	gen. converg.	$\lvert S^*\rvert$	% reducción	Acc GNB (test)	Acc LinearSVC (test)	t_total	t/gen	t/eval
WDBC	30	…	8	73 %	…	…	2.2 s	0.131 s	…
Arrhythmia	279	(tope 100)	93	67 %	…	…	11.1 s	0.111 s	…
Gisette	5000	44	486	90 %	…	…	131.7 s	2.993 s	…

Artefacto: `results/escalabilidad_algoritmica.csv` + figura.

E. Escalabilidad paralela (speed-up por dataset) — Reutilizar el harness de `exp1/2/3` con la configuración de cada dataset: $T(1, 2, 4, 8$ cores) sobre la evaluación de una generación representativa → $S(n)$, $E(n)$, Amdahl. Artefactos: `*_speedup.csv`, `*_speedup.png`.

F. Validación LinearSVC (Tarea 1) — Split `train_test_split(..., test_size=0.3, stratify=y, random_state=42)` antes del GA; el GA busca solo en train; al converger, GNB y LinearSVC entrenan en train con S^* y se miden en test. Alimenta las columnas de validación de la tabla D.

Orden de ejecución

A → F-split → B (re-correr los 3, baratos salvo Gisette) → C (estabilidad) → E (speed-up) → D (consolidación).

Artefactos nuevos

- **Código:** `src/ga_core.py`, `notebooks/07_diversidad_escalabilidad.ipynb`.
- **Por dataset:** `*_diversidad.png`, `*_estabilidad.{csv,png}`, `*_speedup.{csv,png}`, columnas extra en `*_series.csv`.
- **Global:** `results/escalabilidad_algoritmica.csv` + figura, panel comparativo de diversidad.

Coste de reloj estimado

Dominado por Gisette: $ \$22 min (estabilidad, $R = 10$) + $ \$25–30 min (speed-up, incluida la corrida a 1 núcleo) ≈ **menos de 1 hora**. WDBC y Arrhythmia, minutos.

Tiempos por ejecución individual (1 semilla, `local[*]`): WDBC ≈ 2.2 s · Arrhythmia ≈ 11 s · Gisette ≈ 132 s.

Referencias

- Ambroise, C., & McLachlan, G. J. (2002). Selection bias in gene extraction on the basis of microarray gene-expression data. *PNAS*, 99(10), 6562-6566.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.
- Kohavi, R., & John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1-2), 273-324.
- Kuncheva, L. I. (2007). A stability index for feature selection. *Proc. 25th IASTED Int. Conf. on Artificial Intelligence and Applications*, 421-427.
- Siedlecki, W., & Sklansky, J. (1989). A note on genetic algorithms for large-scale feature selection. *Pattern Recognition Letters*, 10(5), 335-347.